ClearURL: Explainable AI-Based Phishing URL Detection System

Iteration 1 Run Sheet and Demo Guide

FIT5238 Group 15 | August, 2026

Public Deployment

The Iteration 1 system is deployed to Alibaba Cloud ECS and can be opened directly at:

http://120.26.118.110/

No local installation is required for the online demonstration. The system uses the live backend and the deployed frontend bundle.

Submission Entry Point

The links below provide direct access to the deployed system and the supporting project evidence.

Moodle upload	This PDF file
Public system	Open the deployed ClearURL system
GitLab repository	Open the GitLab repository
Live Trello board	Open the live Trello board
Shared folder	Open the shared Google Drive folder

Contents

1 Part I: Run and Reproduce the System **2**

1.1 Option A: Open the Deployed System . . . 2
1.2 Option B: Reproduce the System Locally . . . 2
1.3 Start the Backend . . . 2
1.4 Start the Frontend . . . 3
1.5 Validate the Reproduction . . . 3
1.6 Local Recovery Checks . . . 3

2 Part II: Demonstration **3**

2.1 Demonstration Accounts . . . 3
2.2 Employee URL Analysis . . . 3
2.3 Suspicious Website Reporting . . . 3
2.4 Admin Console Review . . . 4
2.5 Data Persistence Demonstration . . . 4
2.6 Demonstration Checklist . . . 4

3 Supporting Evidence **4**

Iteration 1 Evidence Map

Assessment requirement	Primary evidence	Location
Working system and demo	Deployed app and demo steps	Public deployment
Code repository	Source code, tests, README	GitLab main branch
Planning and backlog	User stories, task cards, status	Live Trello board
UX and technical choices	Architecture, APIs, AI integration, deployment	Shared folder - Technical Documentation
Continuous testing	Test record, commands, model results	Shared folder - Test Record
MVP scope and acceptance	User stories and acceptance criteria	Shared folder - User Stories

1 Part I: Run and Reproduce the System

This part provides two ways to reproduce ClearURL. The preferred method is to open the deployed system. The local method is provided as a reproducible backup and development setup.

1.1 Option A: Open the Deployed System

The Iteration 1 system is deployed to Alibaba Cloud ECS:

http://120.26.118.110/

No local installation is required when using this address. The deployed frontend communicates with the live backend and uses the trained model artifact installed on the server.

1.2 Option B: Reproduce the System Locally

The local environment requires:

- Node.js `>= 22`
- Python `3.8+`
- `pip`
- the complete GitLab repository, including `data/` and `artifacts/`

1.3 Start the Backend

From the project root directory, install the backend dependencies and start the Starlette API:

```
python -m pip install -r requirements-backend.txt
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

The backend should be available at `http://127.0.0.1:8000`. The trained model must be present at `artifacts/phishing_detection_model.joblib`.

1.4 Start the Frontend

Open a second terminal in the project root:

```
npm install
npm run dev
```

The Vite development server is normally available at `http://127.0.0.1:5173`. During local development, Vite proxies `/api/` requests to the backend on port `8000`.

1.5 Validate the Reproduction

Run the automated backend tests and verify that the production frontend build completes:

```
python -m pytest -q
npm run build
```

The expected result is that the backend tests pass and the frontend build completes without TypeScript or Vite errors.

1.6 Local Recovery Checks

If the backend does not start, confirm that the dependency installation completed, the `data/` directory is present, and the trained model artifact exists. If analysis does not work from the frontend, confirm that port `8000` is running and that the browser session is authenticated. The admin console additionally requires an authenticated admin session.

2 Part II: Demonstration

2.1 Demonstration Accounts

- Employee: `employee@clearurl.local` / `employee123`
- Admin: `admin@clearurl.local` / `admin123`

2.2 Employee URL Analysis

1. Open the deployed ClearURL system, or open the local frontend.
2. Select `Employee` and sign in with the employee account.
3. Paste a complete URL into the analysis field.
4. Select `Analyse URL`.

Expected result:

- the employee workspace opens after authentication
- the system validates and normalises the URL
- the backend retrieves available webpage content before feature extraction
- the result displays a safety score, risk category, explanations, and recommendations

2.3 Suspicious Website Reporting

1. Select `Report this website` on the result page.
2. Enter an optional explanation or note.
3. Submit the report.

Expected result:

- the report is accepted by the backend
- the report is stored in the feedback database
- the submitted case can be reviewed in the admin console

2.4 Admin Console Review

1. Sign out from the employee session.
2. Select `Admin` and sign in with the admin account.
3. Review `All recent scans`.
4. Review `Recent suspicious cases`.
5. Review the risk distribution, model versions, and evaluation records shown by the console.

Expected result:

- only the admin role can access the administrative data
- newly analysed URLs appear in the scan history
- submitted reports appear as review cases
- the console presents stored monitoring and model-evaluation data

2.5 Data Persistence Demonstration

The demonstration can verify that an analysis is recorded without adding the URL to the training dataset. Analysis records are written to `data/prediction_logs.sqlite3`; submitted reports are written to `data/clearurl-feedback.sqlite3`. Authentication data and model records remain in their separate databases.

2.6 Demonstration Checklist

1. the deployed URL is reachable
2. employee login works
3. URL analysis returns a result
4. the result includes the score, category, reasons, and recommendations
5. a suspicious website report can be submitted
6. admin login works
7. the new scan and report are visible in the admin console
8. automated tests and the frontend build have been checked

3 Supporting Evidence

The following files provide supporting evidence for the Iteration 1 demonstration:

- `README.md`
- `docs/TEST_RECORD.md`
- `docs/MODEL_TEST_RESULTS.md`

- `docs/ITERATION1_ACCEPTANCE_CRITERIA.md`